

Chameleon

One codebase serving many separately branded consumer front-ends — where adding a new client brand is a database row instead of a copy of the project.

LIVE DEMO

chameleon-gray.vercel.app

SOURCE CODE

github.com/adilet0212/chameleon

WHAT IT DOES

The demo runs three storefronts — a coffee roaster, a car dealership and a fitness studio. Each has its own colours, typefaces, page layout and web addresses, and a button on screen switches between them instantly.

All three are the same application. Nothing about any individual brand is written into the code; the differences are stored in a database and applied when the page is requested. That means a company running many client brands can add another one without duplicating and re-maintaining the software.

All three brands are invented for this project. Photography is free-licence stock; some frames incidentally show real production vehicles. No affiliation is implied.

MEASURED — DATABASE INDEX

Product lookup across **15,036 rows**, with and without a composite index on (`tenantId`, `slug`).

Query time in database	Without	With
50th percentile	0.782 ms	0.041 ms
95th percentile	1.001 ms	0.048 ms

19x faster at the 50th percentile. Query plan changes from scanning and discarding 5,011 rows to a direct index lookup. Measured, not estimated — 350 timed runs plus 120 EXPLAIN ANALYZE samples against Neon Postgres 17. Full output is in the repository.

HOW IT IS BUILT

- **Next.js 16** (App Router, server components), TypeScript
- **PostgreSQL** via Prisma — normalised schema, brand data keyed by tenant
- **Tailwind v4** — one design system, per-brand design tokens
- **Playwright** — 18 end-to-end tests on desktop and mobile, all passing
- Deployed on **Vercel**
- Lighthouse mobile — **100** accessibility, best practices and SEO; 85 performance

TWO TESTS THAT MATTER MOST

One brand's page can never render another brand's data, and one brand's item address returns "not found" under a different brand. Both are enforced by the database schema, and both are checked automatically on every run.

WHY I BUILT IT

Shipping high-craft branded experiences for many different clients out of one team is a real engineering problem, and the tempting solution — fork the codebase per client — is the one that hurts two years later. I wanted to build the version that does not fork.

I built it in one evening specifically for this conversation, and I would rather show something narrow and finished, with numbers I actually measured, than something broad and half-working.

